

Python Tip: `dict.get()`

Want to read a dictionary key without raising an error?

`dict.get()` is helpful—but it can also be dangerous.

Let's learn when to use it and when **not** to!



Missing Keys

Bracket access raises a `KeyError` when a key is missing.

```
user = {"name": "Ada", "role": "admin"}

print(user["name"])

# This raises a KeyError
print(user["email"])
```

Ada

```
-----
KeyError                                Traceback (most recent call last)
Cell In[1], line 6
      2
      3 print(user["name"])
      4
      5 # This raises a KeyError
----> 6 print(user["email"])

KeyError: 'email'
```

Using `.get()`

`.get()` returns `None` instead of raising a `KeyError`.

```
user = {"name": "Ada", "role": "admin"}  
  
print(user.get("name"))  
print(user.get("email"))
```

```
Ada  
None
```

This is useful when a key is truly optional.

Providing a Default

Pass a second argument to choose the fallback value.

```
settings = {"theme": "dark"}

theme = settings.get("theme", "light")
language = settings.get("language", "en")

print(theme)
print(language)
```

dark

en

Optional vs. Required

Use `.get()` for optional data. Use brackets for required data.

```
user = {"name": "Ada"}
```

```
# Optional: a fallback makes sense
```

```
nickname = user.get("nickname", user["name"])
```

```
# Required: fail loudly if malformed
```

```
name = user["name"]
```

The Silent Bug

A default can make malformed data look valid.

```
order = {"item": "book"} # Missing required price!  
  
# Silently turns bad data into a free order  
price = order.get("price", 0)  
tax = price * 0.08  
total = price + tax  
  
print(total)
```

0.0

Sometimes a `KeyError` is exactly what you need.

Wrap-Up

AI coding agents often default to `dict.get()` over `dict["key"]`.

That looks defensive, but it can introduce **silent bugs** when required data is malformed.

Check whether each key is truly optional. If it is required, let your code fail loudly!

Follow me for more tips.

Shep Bryan IV